O que é o Insertion Sort?

- É um algoritmo de ordenação que constrói o vetor ordenado **um elemento por vez**, inserindo cada novo elemento na posição correta entre os já ordenados.
- Em sua versão recursiva, a cada execução parcial o algoritmo:
  - **primeiro** ordena recursivamente todo o trecho **exceto o último elemento**;
  - **depois**, insere esse último elemento na posição correta dentro do trecho já ordenado, deslocando os maiores para abrir espaço: custo $\Theta(n)$ no pior caso.

Analogia útil

É como organizar um baralho na mão: você pega uma carta nova e a insere na posição correta entre as cartas que já estão ordenadas.

Pseudocódigo do Insertion Sort (Recursiva)

Universidade de Fortaleza

Entrada: vetor v, índices *ini* e *fim*
Saída : v ordenado entre as posições *ini* e *fim*

```
1  função insertionSort(v, ini, fim):
2      if ini ≥ fim then
3          return ;                          // condição de parada
4      insertionSort(v, ini, fim − 1) ;   // ordena os anteriores
5      chave ← v[fim];
6      j ← fim − 1;
7      while j ≥ ini e v[j] > chave do
8          v[j + 1] ← v[j];
9          j ← j − 1;
10     v[j + 1] ← chave;
```

Pseudocódigo do Insertion Sort (Iterativa)

Entrada: vetor v, índices *ini* e *fim*
Saída : v ordenado entre as posições *ini* e *fim*

```
1 função insertionSortIterativo(v, ini, fim):
2     for i ← ini + 1 to fim do
3         chave ← v[i];
4         j ← i − 1;
5         while j ≥ ini e v[j] > chave do
6             v[j + 1] ← v[j];
7             j ← j − 1;
8         v[j + 1] ← chave;
```

Mesma ideia da versão recursiva, mas com um **laço externo** percorrendo os elementos e um **laço interno** (o `while`) fazendo a inserção.

Exemplo passo a passo

Vamos simular o Insertion Sort no mesmo vetor reduzido usado no Bubble Sort, para facilitar a comparação:

$$v = [\,5,\ 2,\ 8,\ -3,\ 9,\ 1\,]$$

Em cada passagem, um novo elemento é inserido no trecho já ordenado:

azul — trecho já **ordenado** (à esquerda);

vermelho — elemento **sendo inserido** nessa passagem.

Cada passagem mostra o vetor **antes** → **depois** da inserção.

$$[\ 5\ ,\ 2\ ,\ 8,\ -3,\ 9,\ 1\ ] \rightarrow [\ 2\ ,\ 5\ ,\ 8,\ -3,\ 9,\ 1\ ]$$

$2 < 5$: desloca o 5 uma posição à direita e insere o 2 no início.

$$[\ 2\ ,\ 5\ ,\ 8\ ,\ -3,\ 9,\ 1\ ] \rightarrow [\ 2\ ,\ 5\ ,\ 8\ ,\ -3,\ 9,\ 1\ ]$$

$8 > 5$: nenhum deslocamento é necessário, o 8 já está na posição correta.

[2 , 5 , 8 , -3 , 9, 1] → [-3 , 2 , 5 , 8 , 9, 1]

–3 é menor que todos os elementos ordenados: desloca 8, 5 e 2, e insere –3 no início.

[-3 , 2 , 5 , 8 , 9 , 1] → [-3 , 2 , 5 , 8 , 9 , 1]

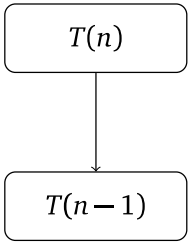
$9 > 8$: nenhum deslocamento é necessário, o 9 já está na posição correta.

[-3 , 2 , 5 , 8 , 9 , 1] → [-3 , 1 , 2 , 5 , 8 , 9]

1 desloca 9, 8, 5 e 2, e é inserido logo após o −3.

Vetor totalmente ordenado:

[−3, 1, 2, 5, 8, 9]



- Apenas **uma** chamada recursiva é feita, sobre o trecho **sem o último elemento**.
- Diferente do Bubble Sort e da Ordenação por Seleção, aqui a recursão acontece **antes** do trabalho: primeiro ordena-se o restante, depois insere-se o elemento atual.
- A inserção pode exigir deslocar até todos os elementos já ordenados: custo $\Theta(n)$ no pior caso.

Construindo a relação de recorrência

A relação de recorrência é usada para analisar a **versão recursiva**, considerando o **pior caso**. A primeira etapa para identificar a classe de complexidade do Insertion Sort é escrever essa relação:

Formulação inicial

$$T(n) = T(n-1) + \Theta(n)$$

Construindo a relação de recorrência

A relação de recorrência é usada para analisar a **versão recursiva**, considerando o **pior caso**. A primeira etapa para identificar a classe de complexidade do Insertion Sort é escrever essa relação:

Formulação inicial

$$T(n) = T(n-1) + \Theta(n)$$

Simplificando:

Forma simplificada

$$T(n) = T(n-1) + n$$

onde $n = fim - ini + 1$ (o tamanho do trecho a ordenar).

$$T(n) = \underbrace{T(n-1)}_{\text{chamada recursiva}} + \underbrace{n}_{\text{inserção}}$$

$T(n-1)$

Representa a **única chamada recursiva**, que ordena o trecho com **um elemento a menos** (todos exceto o último).

$+n$

Representa o custo de **inserir** o último elemento na posição correta.

- No **pior caso** (elemento deve ir para o início), é preciso deslocar todos os $n-1$ elementos já ordenados;
- Por isso, custo **linear**: $\Theta(n)$.

$$T(n) = T(n-1) + n$$

Atenção

Essa é **a mesma forma de recorrência** do Bubble Sort e da Ordenação por Seleção: não se encaixa no Teorema Mestre e o custo fora da chamada recursiva **muda** a cada nível.

Vamos resolvê-la pela **árvore de recursão**.

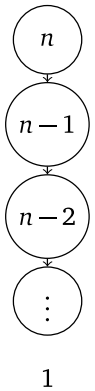
Vamos ilustrar a árvore de recursão gerada por

$$T(n) = T(n-1) + n$$

Características dessa árvore:

- É uma **cadeia** (um nó por nível), pois há apenas **1 chamada recursiva** por passo;
- Cada nó filho reduz a entrada do nó pai em **apenas 1** $\rightarrow$ resume-se em $T(n-1)$;
- O custo de cada nó é proporcional ao tamanho da sua entrada, e por isso **diminui** a cada nível $\rightarrow$ resume-se no termo $+n$.

Árvore de Recursão do Insertion Sort



1

Nível 0:	custo n
Nível 1:	custo $n-1$
Nível 2:	custo $n-2$
$\vdots$	$\vdots$
Nível h:	custo 1

A cada nível, o trecho a ordenar (e, no pior caso, a deslocar) é **um elemento menor**, a mesma estrutura vista no Bubble Sort e na Ordenação por Seleção.

Calculando a altura da árvore

A árvore para de crescer quando o tamanho do trecho chega a 0 (condição de parada `ini >= fim`):

$$n - h = 0$$

Calculando a altura da árvore

A árvore para de crescer quando o tamanho do trecho chega a 0 (condição de parada `ini >= fim`):

$$n - h = 0$$

Isolando h:

$$h = n$$

A altura da árvore é **proporcional a** n, e o custo por nível **não é o mesmo** em todos os níveis.

Como o custo de cada nível **muda** a cada nível, precisamos **somar** os custos de todos os níveis:

$$f(n) = n + (n-1) + (n-2) + \cdots + 2 + 1$$

Como o custo de cada nível **muda** a cada nível, precisamos **somar** os custos de todos os níveis:

$$f(n) = n + (n-1) + (n-2) + \cdots + 2 + 1$$

Essa é a soma dos n primeiros números naturais:

Tempo total

$$f(n) = \frac{n \cdot (n+1)}{2} = \frac{n^2 + n}{2}$$

O termo dominante é n^2, logo $f(n) = \Theta(n^2)$.

$$T(n) = \Theta(n^2)$$

- Resolvendo a recorrência $T(n) = T(n-1) + n$ pela árvore de recursão, obtemos um tempo de execução **quadrático** no pior caso;
- No **pior caso** (vetor em ordem decrescente), cada inserção desloca todos os elementos já ordenados: $\Theta(n^2)$;
- No **melhor caso** (vetor já ordenado), cada elemento é inserido sem deslocamentos, apenas com uma comparação: $\Theta(n)$;
- Diferente do Bubble Sort e da Seleção, o Insertion Sort é **sensível à ordem de entrada**, funciona bem em vetores quase ordenados.

Analisando a versão iterativa

A versão iterativa não é analisada por recorrência, mas o raciocínio é direto: basta contar quantas vezes os laços aninhados executam **no pior caso**.

- O laço **externo** executa $n-1$ vezes (uma para cada elemento a inserir, do segundo ao último);
- Para o i-ésimo elemento, o laço **interno** pode deslocar até $i-1$ elementos no pior caso;
- Somando todas as iterações do laço interno:

Analisando a versão iterativa

A versão iterativa não é analisada por recorrência, mas o raciocínio é direto: basta contar quantas vezes os laços aninhados executam **no pior caso**.

- O laço **externo** executa $n-1$ vezes (uma para cada elemento a inserir, do segundo ao último);
- Para o i-ésimo elemento, o laço **interno** pode deslocar até $i-1$ elementos no pior caso;
- Somando todas as iterações do laço interno:

Tempo total (pior caso)

$$f(n) = 1 + 2 + \cdots + (n-1) = \frac{n(n-1)}{2} \Rightarrow T(n) = \Theta(n^2)$$

Mesma complexidade de tempo da versão recursiva, mas sem o custo de **espaço** das chamadas na pilha de recursão.

	Recursiva	**Iterativa**
Tempo (pior caso)	$\Theta(n^2)$	$\Theta(n^2)$
Tempo (melhor caso)	$\Theta(n)$	$\Theta(n)$
Espaço extra	$\Theta(n)$ (pilha)	$\Theta(1)$
Análise	recorrência → árvore	laços aninhados

- As duas versões fazem o **mesmo número de comparações e deslocamentos**, em qualquer caso;
- A diferença está no uso de memória: cada chamada recursiva ocupa espaço na pilha, enquanto a iterativa reutiliza as mesmas variáveis.

- O Insertion Sort constrói o vetor ordenado **um elemento por vez**, inserindo cada novo elemento na posição correta entre os já ordenados.
- **Versão recursiva**: $T(n) = T(n-1) + n$ (pior caso), resolvida pela árvore de recursão (altura $h = n$, custos $n, n-1, \ldots, 1$).
- **Versão iterativa**: laço externo + laço interno de deslocamento, somando até $\frac{n(n-1)}{2}$ operações.
- Complexidade final: $\Theta(n^2)$ no pior caso, $\Theta(n)$ no melhor caso (vetor já ordenado).
- A versão iterativa evita o custo de espaço extra da pilha de recursão.

Comparando os três algoritmos quadráticos

Algoritmo	Melhor	Médio	Pior	Trocas	Estável
Seleção	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n)$	Não
Trocas	$\Theta(n)^*$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	Sim
Inserção	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	Sim

* com a otimização de parada antecipada (nenhuma troca em uma passagem).

- Todos usam $\Theta(1)$ de espaço extra (versões iterativas);
- Um algoritmo é **estável** se mantém a ordem relativa de chaves iguais;
- Na prática, a Inserção é a mais rápida das três para n pequeno – por isso bibliotecas reais a usam dentro do Quicksort quando o subvetor fica curto.

Dúvidas?

Perguntas? Vamos programar?



Aula 09 – 29/09

Ordenação por intercalação (Mergesort) e Quicksort

O que é o MergeSort?

- É um algoritmo de ordenação baseado na estratégia **dividir para conquistar** (*divide and conquer*).
- A cada execução parcial, o algoritmo:
  - faz **duas chamadas recursivas**, dividindo o tamanho da entrada pela metade em cada uma;
  - executa uma rotina de **merge**, responsável por unir duas sequências já ordenadas em uma única sequência ordenada.
- O custo dessa rotina de merge é dado por $\Theta(n)$.

Entrada: vetor v, índices *ini* e *fim*
Saída : v ordenado entre as posições *ini* e *fim*

```
1 função mergeSort(v, ini, fim):
2     if ini ≥ fim then
3         return ;                          // condição de parada
4     meio ← ⌊(ini + fim)/2⌋;
5     mergeSort(v, ini, meio);
6     mergeSort(v, meio + 1, fim);
7     merge(v, ini, meio, fim);
```

Pseudocódigo do Merge

Entrada: vetor v, índices ini, $meio$, fim
Saída : v com o intervalo $[ini, fim]$ ordenado

```
1  função merge(v, ini, meio, fim):
2      esq ← cópia de v[ini ... meio];
3      dir ← cópia de v[meio + 1 ... fim];
4      i ← 0; j ← 0; k ← ini;
5      while i < esq.length e j < dir.length do
6          if esq[i] ≤ dir[j] then
7              v[k] ← esq[i]; i ← i + 1;
8          else
9              v[k] ← dir[j]; j ← j + 1;
10         k ← k + 1;
11     copia os elementos restantes de esq e dir para v;
```

Percorre os dois subvetores **uma única vez**, comparando seus elementos $\Rightarrow$ custo $\Theta(n)$.

Exemplo passo a passo

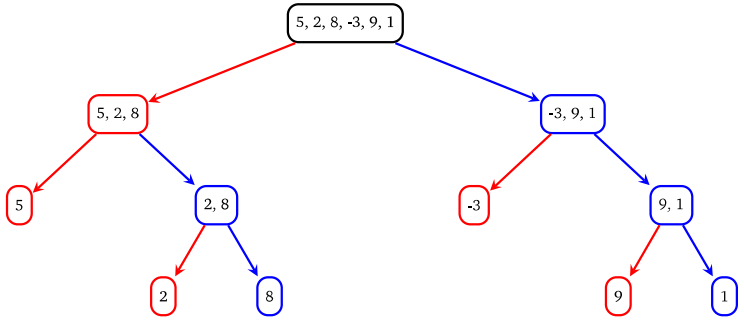
Vamos simular o MergeSort no mesmo vetor reduzido usado nos algoritmos anteriores:

$$v = [\,5,\ 2,\ 8,\ -3,\ 9,\ 1\,]$$

O algoritmo tem duas fases bem distintas, que vamos visualizar em árvore:

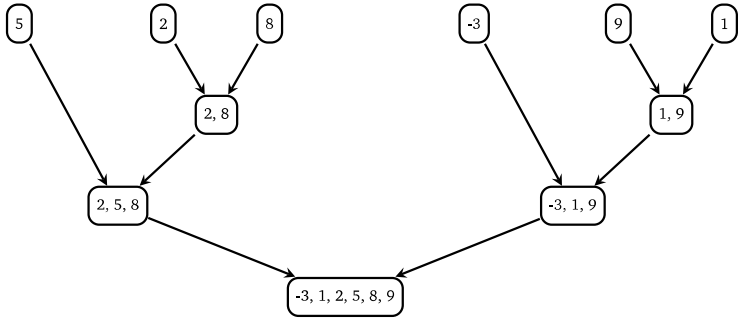
Divisão — o vetor é dividido ao meio repetidamente até restarem elementos únicos;

Intercalação (Merge) — os pedaços já ordenados são combinados de volta, dois a dois, até formar o vetor final.

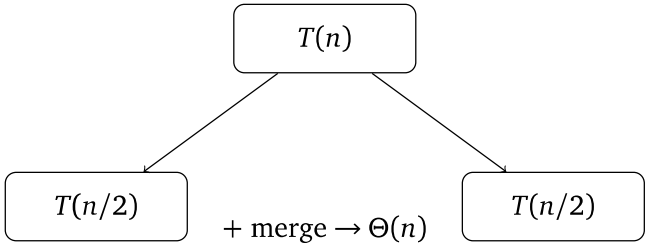


A divisão para quando cada trecho tem apenas 1 elemento (trivialmente ordenado).

Fase de Intercalação (Merge)



Cada `merge` combina dois trechos já ordenados em tempo linear. O resultado final está ordenado: $[-3, 1, 2, 5, 8, 9]$.



- Cada chamada recursiva resolve **metade** do problema original.
- Depois que as duas metades estão ordenadas, o **merge** as combina em tempo linear.

A primeira etapa para identificar a classe de complexidade do MergeSort é escrever sua **relação de recorrência**:

Formulação inicial

$$T(n) = T(n/2) + T(n/2) + \Theta(n)$$

A primeira etapa para identificar a classe de complexidade do MergeSort é escrever sua **relação de recorrência**:

Formulação inicial

$$T(n) = T(n/2) + T(n/2) + \Theta(n)$$

Simplificando:

Forma simplificada

$$T(n) = 2 \cdot T(n/2) + n$$

onde $n = v.\text{length}$ (o tamanho do vetor de entrada).

Interpretando cada termo

$$T(n) = \underbrace{2 \cdot T(n/2)}_{\text{chamadas recursivas}} + \underbrace{n}_{\text{merge}}$$

$2 \cdot T(n/2)$

Representa as **duas chamadas recursivas**, em que a entrada é dividida pela metade em cada uma delas.

$+n$

Representa o custo da função **merge**, que une duas sequências já ordenadas em uma única sequência ordenada.

- Não é necessário saber *como* o merge é feito internamente;
- Basta saber que essa etapa tem custo **linear**.

$$T(n) = 2 \cdot T\left(\frac{n}{2}\right) + n$$

- $a = 2 \rightarrow$ número de subproblemas gerados a cada chamada;
- $b = 2 \rightarrow$ fator pelo qual o tamanho da entrada é dividido;
- $f(n) = n \rightarrow$ custo do trabalho realizado fora das chamadas recursivas (o merge).

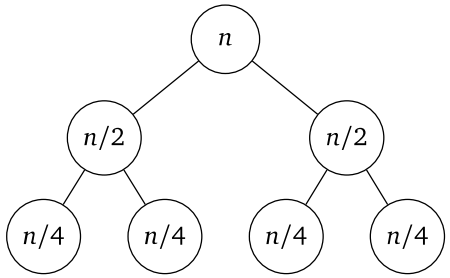
Essa relação está no formato padrão exigido pelo **Teorema Mestre**, que pode ser usado para resolvê-la e obter a complexidade assintótica do algoritmo.

Vamos ilustrar a árvore de recursão gerada por

$$T(n) = 2 \cdot T(n/2) + n$$

Diferenças em relação à árvore do fatorial:

- A árvore agora é **binária** → o custo de um nível é a **soma dos custos de todos os nós** desse nível;
- As arestas representam as **duas chamadas recursivas** de cada passo;
- Cada **nó filho** tem a entrada do **nó pai dividida pela metade** → resume-se em $2 \cdot T(n/2)$;
- Cada nó tem tempo de execução **linear** em relação ao tamanho da sua entrada → resume-se no termo $+n$.



Nível 0:	custo n
Nível 1:	custo $n/2 + n/2 = n$
Nível 2:	custo $4 \cdot n/4 = n$
$\vdots$	$\vdots$
Nível h:	custo n

Cada nível, do topo até a base da árvore, tem **o mesmo custo total:** n.

Calculando a altura da árvore

A árvore para de crescer quando o tamanho da entrada de um nó chega a 1 (condição de parada `ini >= fim`):

$$\frac{n}{2^h} = 1$$

Calculando a altura da árvore

A árvore para de crescer quando o tamanho da entrada de um nó chega a 1 (condição de parada `ini >= fim`):

$$\frac{n}{2^h} = 1$$

Isolando 2^h:

$$2^h = n$$

Calculando a altura da árvore

A árvore para de crescer quando o tamanho da entrada de um nó chega a 1 (condição de parada `ini >= fim`):

$$\frac{n}{2^h} = 1$$

Isolando 2^h:

$$2^h = n$$

Aplicando $\log_2$ nos dois lados:

$$h \cdot \log_2 2 = \log_2 n$$

Calculando a altura da árvore

A árvore para de crescer quando o tamanho da entrada de um nó chega a 1 (condição de parada `ini >= fim`):

$$\frac{n}{2^h} = 1$$

Isolando 2^h:

$$2^h = n$$

Aplicando $\log_2$ nos dois lados:

$$h \cdot \log_2 2 = \log_2 n$$

Como $\log_2 2 = 1$, simplificando:

$$h = \log_2 n$$

Para obter o tempo total, somamos os custos **de todos os níveis** da árvore.

- Cada nível tem custo n (ex.: $2 \cdot n/2$, $4 \cdot n/4$, $8 \cdot n/8, \ldots$);
- A árvore tem altura $h = \log_2 n$, ou seja, $\log_2 n$ níveis;
- Somar n um total de $\log_2 n$ vezes:

Para obter o tempo total, somamos os custos **de todos os níveis** da árvore.

- Cada nível tem custo n (ex.: $2 \cdot n/2$, $4 \cdot n/4$, $8 \cdot n/8, \ldots$);
- A árvore tem altura $h = \log_2 n$, ou seja, $\log_2 n$ níveis;
- Somar n um total de $\log_2 n$ vezes:

Tempo total

$$f(n) = \underbrace{n + n + \cdots + n}_{\log_2 n \text{ vezes}} = n \cdot \log_2 n$$

Essa multiplicação só é válida porque **todo nível tem o mesmo custo** n.

$$T(n) = \Theta(n \cdot \log n)$$

- Resolvendo a recorrência $T(n) = 2T(n/2) + n$ pela árvore de recursão, chegamos à mesma conclusão que o Teorema Mestre indicaria;
- O MergeSort tem complexidade de tempo $\Theta(n \log n)$ em todos os casos (melhor, médio e pior);
- Essa é uma classe de complexidade melhor do que algoritmos $\Theta(n^2)$, como Bubble Sort e Insertion Sort.

- O MergeSort segue o paradigma de **dividir para conquistar**.
- A cada chamada, o problema é dividido em **2 subproblemas de tamanho** $n/2$.
- A combinação das soluções (merge) custa $\Theta(n)$.
- Relação de recorrência obtida:

$$T(n) = 2T(n/2) + n$$

- Pela **árvore de recursão**: altura $h = \log_2 n$, custo n por nível $\Rightarrow$

$$T(n) = n \cdot \log_2 n$$

- Complexidade final: $T(n) = \Theta(n \log n)$.

Dúvidas?

Perguntas?



O que é o QuickSort?

- É um algoritmo de ordenação baseado em **dividir para conquistar**, assim como o MergeSort.
- A cada execução parcial, o algoritmo:
  - escolhe um elemento como **pivô** e **particiona** o vetor: menores que o pivô à esquerda, maiores à direita — custo $\Theta(n)$;
  - faz **duas chamadas recursivas**, uma para cada lado da partição.
- Diferente do MergeSort, **não há uma etapa de merge**: ao final das duas chamadas recursivas, o vetor já está ordenado.

Diferença importante em relação ao MergeSort

As duas partes geradas pela partição **não têm necessariamente o mesmo tamanho** — isso depende da escolha do pivô, e afeta diretamente a complexidade do algoritmo.

Pseudocódigo do QuickSort (Recursiva)

Universidade de Fortaleza

Entrada: vetor v, índices *ini* e *fim*
Saída : v ordenado entre as posições *ini* e *fim*

```
1 função quickSort(v, ini, fim):
2     if ini < fim then
3         p ← particionar(v, ini, fim);
4         quickSort(v, ini, p − 1);
5         quickSort(v, p + 1, fim);
```

Entrada: vetor v, índices *ini* e *fim*
Saída : posição final do pivô

```
1 função particionar(v, ini, fim):
2     pivo ← v[fim];
3     i ← ini − 1;
4     for j ← ini to fim − 1 do
5         if v[j] ≤ pivo then
6             i ← i + 1;
7             troca v[i] e v[j];
8     troca v[i + 1] e v[fim];
9     return i + 1;
```

Percorre o vetor **uma única vez**, movendo os elementos $\leq pivo$ para a esquerda $\Rightarrow$ custo $\Theta(n)$.

Pseudocódigo do QuickSort (Iterativa)

Entrada: vetor v, índices *ini* e *fim*

```
1 função quickSortIterativo(v, ini, fim):
2     pilha ← vazia;
3     empilha(ini, fim);
4     while pilha não vazia do
5         (ini, fim) ← desempilha();
6         if ini < fim then
7             p ← particionar(v, ini, fim);
8             empilha(ini, p − 1);
9             empilha(p + 1, fim);
```

Substitui a pilha de chamadas recursivas por uma **pilha explícita** de intervalos (*ini*,*fim*) a processar.

Vamos simular o QuickSort no mesmo vetor reduzido dos exemplos anteriores, usando o **último elemento** como pivô em cada partição:

$$v = [\,5,\ 2,\ 8,\ -3,\ 9,\ 1\,]$$

amarelo — o pivô sendo posicionado **nesta** etapa;

azul — pivôs de etapas **anteriores**, já em sua posição final.

Pivô escolhido: **último elemento** do vetor.

$$[\ 5,\ 2,\ 8,\ -3,\ 9,\ \boxed{1}\ ] \rightarrow [\ -3,\ \boxed{1}\ ,\ 8,\ 5,\ 9,\ 2\ ]$$

O pivô 1 vai para o índice 1. À esquerda, apenas $[-3]$ (já ordenado). À direita, o subvetor $[8, 5, 9, 2]$ ainda precisa ser particionado.

2º particionamento (subvetor $[8, 5, 9, 2]$)

Pivô escolhido: **último elemento** do subvetor.

$$[\ -3,\ 1\ ,\ 8,\ 5,\ 9,\ 2\ ] \rightarrow [\ -3,\ 1\ ,\ 2\ ,\ 5,\ 9,\ 8\ ]$$

O pivô 2 vai para o índice 2. Nenhum elemento é menor que 2 nesse subvetor. À direita, o subvetor $[5, 9, 8]$ ainda precisa ser particionado.

3º particionamento (subvetor $[5, 9, 8]$)

Pivô escolhido: **último elemento** do subvetor.

$$[\ -3,\ 1,\ \boxed{2}\ ,\ 5,\ 9,\ \boxed{8}\ ]\ \rightarrow\ [\ -3,\ 1,\ \boxed{2}\ ,\ 5,\ \boxed{8}\ ,\ 9\ ]$$

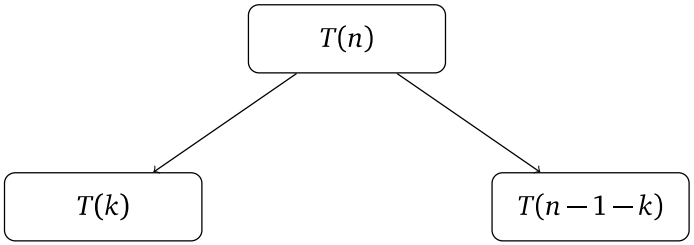
O pivô 8 vai para o índice 4. À esquerda sobra $[5]$ e à direita sobra $[9]$ — ambos com um único elemento, portanto já ordenados.

Todos os subvetores restantes têm no máximo 1 elemento (caso base da recursão): não há mais nada a particionar.

Vetor totalmente ordenado:

$$[\ -3,\ 1,\ 2,\ 5,\ 8,\ 9\ ]$$

Neste exemplo, cada partição isolou apenas **um** elemento à esquerda do pivô — um padrão de partição bem **desbalanceada**, próximo do pior caso discutido mais adiante.



- k é o número de elementos menores que o pivô, isto é, o **tamanho do lado esquerdo** da partição;
- O tamanho dos dois lados depende de **onde o pivô cai** na partição — pode ser qualquer valor entre 0 e $n-1$;
- Antes das chamadas recursivas, o algoritmo paga o custo **linear** da partição.

A primeira etapa para identificar a classe de complexidade do QuickSort é escrever sua **relação de recorrência**:

Formulação geral

$$T(n) = T(k) + T(n-1-k) + \Theta(n)$$

O valor de k importa

Diferente do MergeSort, aqui **não sabemos de antemão** o tamanho dos subproblemas — isso depende da escolha do pivô. Por isso, analisamos **dois casos extremos**: partição balanceada e partição totalmente desbalanceada.

Diferente do MergeSort, no QuickSort o tamanho dos subproblemas depende de **onde o pivô cai** a cada partição. Por isso, vamos considerar **três cenários distintos**:

- **Pior caso**: o pivô é sempre o menor ou o maior elemento;
- **Melhor caso**: o pivô sempre divide a lista ao meio;
- **Caso médio**: a média sobre **todas** as partições possíveis.

Ocorre quando o pivô é sempre o **maior ou menor elemento**, gerando partições totalmente desbalanceadas.

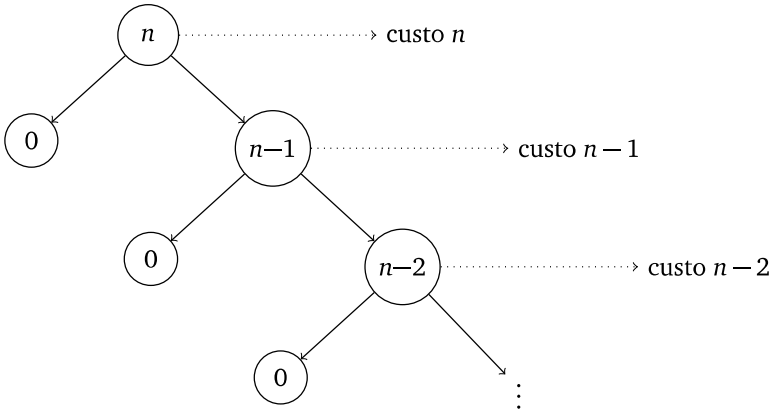
Na primeira chamada, uma lista de tamanho n gera uma lista de tamanho $n-1$ e uma lista **vazia**. Isso nos dá:

Relação de recorrência

$$T(n) = T(n-1) + T(0) + n = T(n-1) + n$$

É **exatamente a mesma recorrência** da Ordenação por Seleção, do Bubble Sort e do Insertion Sort.

Árvore de recursão do pior caso



A cada nível, a partição vazia (0) encerra imediatamente, enquanto o outro lado continua a cadeia — mesma estrutura em cadeia vista nos algoritmos de ordenação anteriores.

Somando os custos de todos os níveis:

$$T(n) = 1 + 2 + 3 + \cdots + (n-1) + n = \sum_{k=1}^{n} k$$

Somando os custos de todos os níveis:

$$T(n) = 1 + 2 + 3 + \cdots + (n-1) + n = \sum_{k=1}^{n} k$$

Resultado

$$T(n) = \frac{n(n+1)}{2} = \frac{n^2 + n}{2}$$

$$T(n) = O(n^2)$$

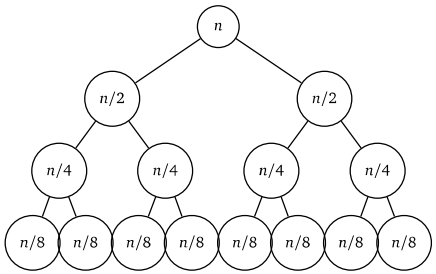
Ocorre quando as duas partições têm **exatamente o mesmo tamanho**: $n/2$ elementos, o pivô, e mais $n/2$ elementos.

Relação de recorrência

$$T(n) = T(n/2) + T(n/2) + n = 2 \cdot T(n/2) + n$$

É **exatamente a mesma recorrência** do MergeSort — o custo n vem da função de partição, assim como no MergeSort vinha da função de merge.

Árvore de recursão do melhor caso



Nível 0:	custo n
Nível 1:	custo n
Nível 2:	custo n
$\vdots$	$\vdots$
Nível h:	custo n

A árvore continua se dividindo até restar 1 elemento por folha:
$\frac{n}{2^h} = 1 \Rightarrow h = \log_2 n.$

Como a altura da árvore é $h = \log_2 n$ e o custo em **cada** nível é n:

$$T(n) = \underbrace{n + n + \cdots + n}_{\log_2 n \text{ vezes}}$$

Como a altura da árvore é $h = \log_2 n$ e o custo em **cada** nível é n:

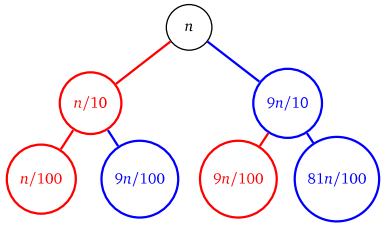
$$T(n) = \underbrace{n + n + \cdots + n}_{\log_2 n \text{ vezes}}$$

Resultado

$$T(n) = O(n \log_2 n)$$

- Uma estratégia empírica comum: escolher o pivô como a **mediana entre o primeiro, o elemento do meio e o último** (regra da mediana de 3) — reduz bastante a chance de partições ruins.
- A grande vantagem do QuickSort é que ele opera no modo **caso médio** praticamente sempre;
- Mesmo com partições consistentemente desiguais — por exemplo, sempre 90%–10% — o algoritmo **ainda é** $O(n \log n)$.

Árvore de recursão para uma partição 90/10



- **Ramo vermelho** (sempre 10%): o **mais curto** da árvore, altura $h_L = \log_{10} n$;
- **Ramo azul** (sempre 90%): o **mais longo** da árvore, altura $h_R = \log_{10/9} n$.

Resolvendo o caso 90/10

- Até o nível h_L, **todo** nível da árvore tem custo n;
- Depois de h_L, o custo por nível é **menor** que n (pois já restam ramos vazios/encerrados);
- Somando todos os níveis (usando a altura do ramo mais longo como limite superior):

Limite superior

$$T(n) \leq c \cdot n \cdot \log_{10/9} n$$

- Até o nível h_L, **todo** nível da árvore tem custo n;
- Depois de h_L, o custo por nível é **menor** que n (pois já restam ramos vazios/encerrados);
- Somando todos os níveis (usando a altura do ramo mais longo como limite superior):

Limite superior

$$T(n) \leq c \cdot n \cdot \log_{10/9} n$$

Como $\log_{10/9} n \approx 6{,}578 \cdot \log_2 n$ (uma constante multiplicativa), concluímos:

$$T(n) = O(n \log n)$$

Mesmo uma partição bem desigual ainda resulta em complexidade $O(n \log n)$.

$$T(n) = O(n \log n) \text{ (melhor e caso médio)}$$

$$T(n) = O(n^2) \text{ (pior caso)}$$

- O QuickSort é, **na prática**, tão rápido quanto o MergeSort e o Heap Sort — porque o caso médio domina;
- O pior caso $O(n^2)$ exige uma escolha consistentemente ruim de pivô (ex.: sempre o menor/maior elemento em um vetor já ordenado);
- Estratégias como a **mediana de três** ou escolher o pivô **aleatoriamente** tornam o pior caso extremamente improvável.

Analisando a versão iterativa

A versão iterativa realiza **exatamente as mesmas partições** que a recursiva — apenas substitui a pilha de chamadas por uma pilha explícita de intervalos.

- As complexidades de tempo são as **mesmas** da versão recursiva: $O(n \log n)$ no melhor caso e no caso médio, $O(n^2)$ no pior caso;
- A vantagem está no **controle do espaço**: com a pilha explícita, é possível empilhar sempre o **lado maior** da partição e continuar imediatamente no **lado menor**, garantindo pilha de tamanho $\Theta(\log n)$ mesmo no pior caso de desbalanceamento.

	Recursiva	**Iterativa**
Tempo (melhor/médio)	$O(n \log n)$	$O(n \log n)$
Tempo (pior caso)	$O(n^2)$	$O(n^2)$
Espaço extra (pior caso)	$\Theta(n)$ (pilha)	$\Theta(n)$ ou $\Theta(\log n)^*$

* com a otimização de empilhar sempre o lado maior da partição.

- As duas versões fazem as **mesmas comparações e trocas**;
- A iterativa permite controlar melhor o uso de memória, algo que a recursiva não oferece diretamente.

- O QuickSort particiona o vetor em torno de um **pivô** e recursa nas duas partes; não há etapa de merge.
- **Pior caso**: $T(n) = T(n-1) + n$ (partição vazia $+ n-1$) $\Rightarrow O(n^2)$.
- **Melhor caso**: $T(n) = 2T(n/2) + n$ (partições iguais, igual ao MergeSort) $\Rightarrow O(n \log n)$.
- **Caso médio**: média sobre todas as partições possíveis, resolvida por manipulação algébrica e telescopagem $\Rightarrow$ também $O(n \log n)$.
- Mesmo com partições desiguais na prática (ex.: 90%–10%), o algoritmo continua $O(n \log n)$ — por isso o QuickSort é tão usado.
- Versões recursiva e iterativa têm a **mesma complexidade de tempo**; a iterativa pode limitar o espaço extra a $\Theta(\log n)$ com a escolha certa de qual lado empilhar.

	MergeSort	QuickSort
Melhor caso	$\Theta(n \log n)$	$\Theta(n \log n)$
Caso médio	$\Theta(n \log n)$	$\Theta(n \log n)$
Pior caso	$\Theta(n \log n)$	$\Theta(n^2)$
Espaço extra	$\Theta(n)$	$\Theta(\log n)$ (pilha)
Estável	Sim	Não
Onde brilha	dados externos, listas	memória interna, cache

Ideia-chave

Na prática, o QuickSort costuma ser mais rápido por ter constantes menores e trabalhar *in place*; o MergeSort oferece a **garantia** de $n \log n$ em todos os casos.

Dúvidas?

Perguntas? Vamos programar?

